field, and then use it to wrap the current logger call in a condition that uses the `Is-Development` extension method. The revised `IndexModel` class is shown in the next listing.

Listing 14.7 The revised `IndexModel` logging only in development

```
public class IndexModel : PageModel
{
    private readonly ICityService _cityService;
    private readonly ILogger<IndexModel> _logger;
    private readonly IHostEnvironment _environment;                          ❶
    public IndexModel(ILogger<IndexModel> logger,
    ➥ ICityService cityService, IHostEnvironment environment)               ❶
    {
        _logger = logger;
        _cityService = cityService;
        _environment = environment;                                          ❶
    }

    public List<City> Cities { get; set; }
    public async Task OnGetAsync()
    {

        Cities = await _cityService.GetAllAsync();
        if (_environment.IsDevelopment())                                    ❷
        {                                                                    ❷
            _logger.LogInformation("{count} Cities retrieved", Cities.Count);❷
        }                                                                    ❷
    }
}
```

❶ Inject the IHostEnvironment service, and assign it to a private field.

❷ Use the IsDevelopment extension method to ensure logging takes place only when the environment is Development.

If you run the application normally and then use a launch profile or the `WebApplicationOptions` to change the environment, you should see that the log is only generated when the environment is `Development`.

NOTE The use of the `IHostEnvironment` extension methods to control logging based on environments is not recommended. Later, you will see how to use the appSettings.json file to control